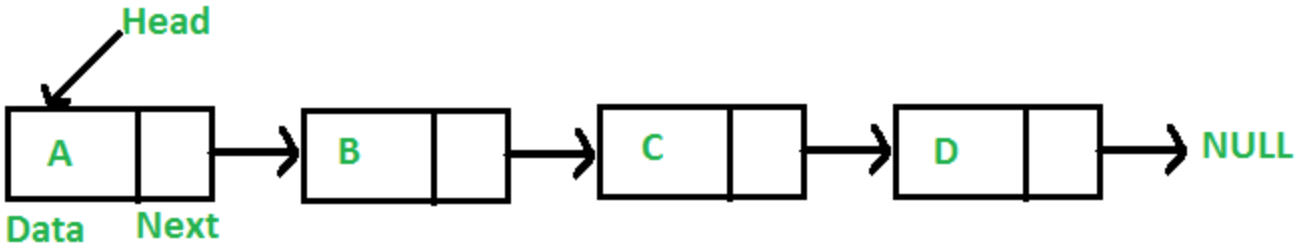
Linked List

By
Shital Dongre
Asst. Prof.
IT & MCA Dept,
VIT, Pune

Linked List

- linked list is a linear data structure
- elements are not stored at contiguous memory locations.
- elements in a linked list are linked using pointers
- linked list consists of nodes where each node contains a data field and a reference(link) to the next node in the list.



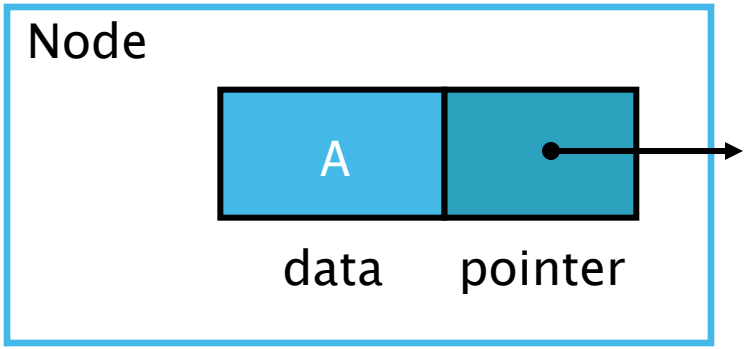
Why Linked List?

Arrays can be used to store linear data of similar types, but arrays have following limitations.
1) The size of the arrays is fixed: So we must know the upper limit on the number of elements in advance. Also, generally, the allocated memory is equal to the upper limit irrespective of the usage.
2) Inserting a new element in an array of elements is expensive, because room has to be created for the new elements and to create room existing elements have to shifted.

- **Advantages over arrays**
  1) Dynamic size
  2) Ease of insertion/deletion
- **Drawbacks**:
  1) Random access is not allowed. We have to access elements sequentially starting from the first node. So we cannot do binary search with linked lists.
  2) Extra memory space for a pointer is required with each element of the list.
  3) Not cache friendly. Since array elements are contiguous locations, there is locality of reference which is not there in case of linked lists.

- A linked list is represented by a pointer to the first node of the linked list. The first node is called head. If the linked list is empty, then value of head is NULL.
  Each node in a list consists of at least two parts:
  1) data
  2) Pointer (Or Reference) to the next node

```
// A linked list node
struct Node
{
  int data;
  struct Node *next;
};
```



```
struct Node
{
  int data;
  struct Node *next;
};
int main()
{
  struct Node* head = NULL;
  struct Node* second = NULL;
  struct Node* third = NULL;

  // allocate 3 nodes in the heap
  head = (struct Node*)malloc(sizeof(struct Node));
  second = (struct Node*)malloc(sizeof(struct Node));
  third = (struct Node*)malloc(sizeof(struct Node));
head->data = 1;
head->next = second;

second->data = 2;
second->next = third;

 third->data = 3;
 third->next = NULL;
return 0;
}
```

Linked list traversal

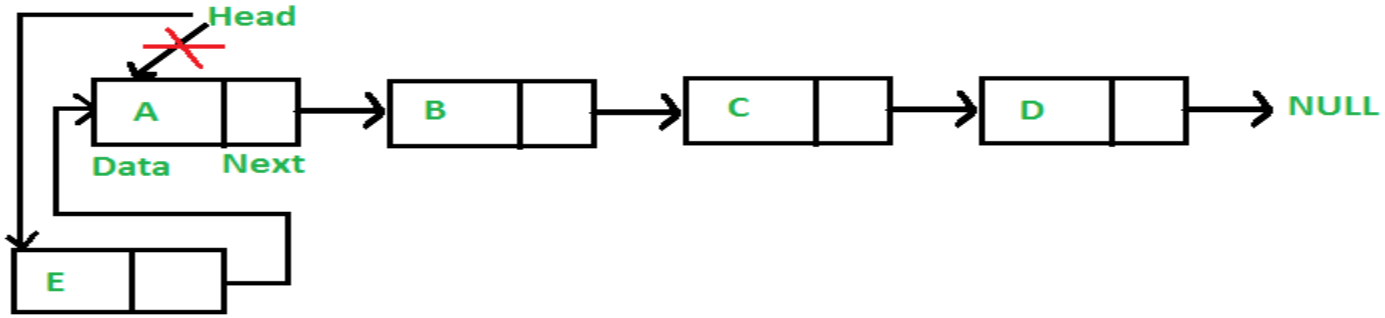
```
void printList(struct Node *n)
{
  while (n != NULL)
  {
     printf(" %d ", n->data);
     n = n->next;
  }
}
.
.
.
printList(head);
```

Insert node

A node can be added in three ways

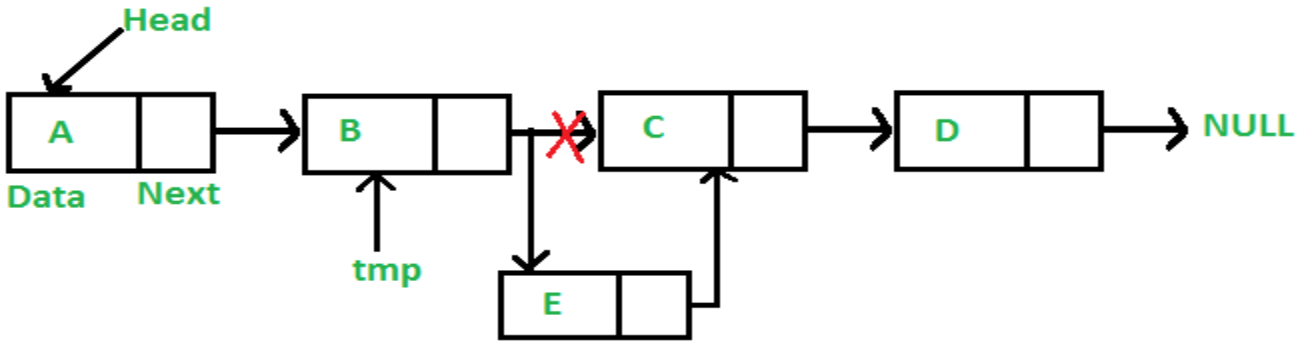
1) At the front of the linked list
2) After a given node.
3) At the end of the linked list.

Insert at front



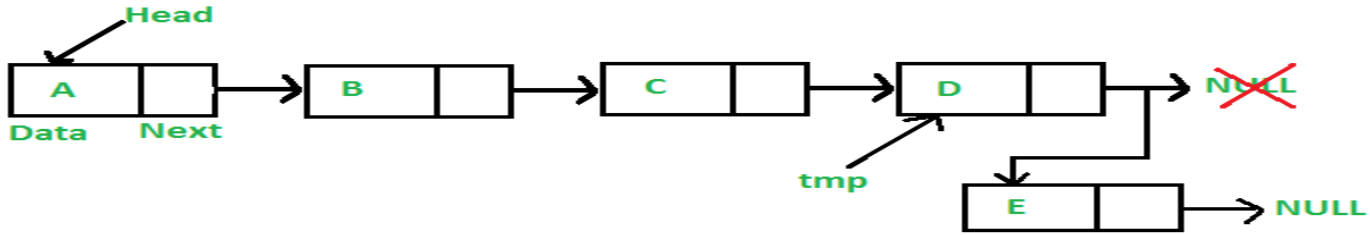

```
void push(struct Node** head_ref, int new_data)
{
    /* 1. allocate node */
    struct Node* new_node = (struct Node*) malloc(sizeof(struct Node));

    /* 2. put in the data  */
    new_node->data  = new_data;

    /* 3. Make next of new node as head */
    new_node->next = (*head_ref);

    /* 4. move the head to point to the new node */
    (*head_ref)    = new_node;
}
```

Insert in between




```
void insertAfter(struct Node* prev_node, int new_data)
{
    /*1. check if the given prev_node is NULL */
    if (prev_node == NULL)
    {
       printf("the given previous node cannot be NULL");
       return;
    }

    /* 2. allocate new node */
    struct Node* new_node =(struct Node*) malloc(sizeof(struct Node));

    /* 3. put in the data  */
    new_node->data  = new_data;

    /* 4. Make next of new node as next of prev_node */
    new_node->next = prev_node->next;

    /* 5. move the next of prev_node as new_node */
    prev_node->next = new_node;
}
```

Insert at last




```
void append(struct Node** head_ref, int new_data)
{
    /* 1. allocate node */
    struct Node* new_node = (struct Node*) malloc(sizeof(struct Node));

    struct Node *last = *head_ref;  /* used in step 5*/

    /* 2. put in the data  */
    new_node->data  = new_data;

    /* 3. This new node is going to be the last node, so make next
         of it as NULL*/
    new_node->next = NULL;

    /* 4. If the Linked List is empty, then make the new node as head */
    if (*head_ref == NULL)
    {
       *head_ref = new_node;
       return;
    }

    /* 5. Else traverse till the last node */
    while (last->next != NULL)
        last = last->next;

    /* 6. Change the next of last node */
    last->next = new_node;
    return;
}
```

Search node

```
void search(struct Node* head, int key)
{
Int i=1;

      struct Node* tmp;
tmp =head;

while (tmp->next != NULL)
{
          if(tmp->data==key)
          {
          printf("node location=%d", i);
          break;
          }
      tmp  = tmp->next;
     i++;

}
```

```
/* Counts no. of nodes in linked list */
int getCount(struct Node* head)
{
    int count = 0;  // Initialize count
    struct Node* current = head;  // Initialize current
    while (current != NULL)
    {
        count++;
        current = current->next;
    }
    return count;
}
```

Delete node

- From front
- From matching key
- Last

```
void deleteNode(struct Node *head, int key)
{
struct Node* temp = *head, *prev;
if (temp != NULL && temp->data == key)
{ *head = temp->next;
free(temp);
return;
}
while (temp != NULL && temp->data != key)
 {
 prev = temp;
temp = temp->next;
}
if (temp == NULL)
 return;
prev->next = temp->next;
free(temp);
}
```

- Delete entire linked list

```
void deleteList(struct Node* head)
{
   /* deref head_ref to get the real head */
   struct Node* current = *head;
   struct Node* next_node;

   while (current != NULL)
   {
       next_node = current->next;
       free(current);
       current = next_node;
   }

   /* deref head_ref to affect the real head back
      in the caller. */
   *head = NULL;
}
```

Reverse list

```
void reverse(struct Node** head)
{
    struct Node* prev   = NULL;
    struct Node* current = *head;
    struct Node* next_node;
    while (current != NULL)
    {
        next_node  = current->next;
        current->next = prev;
        prev = current;
        current = next_node;
    }
    *head = prev;
}
```

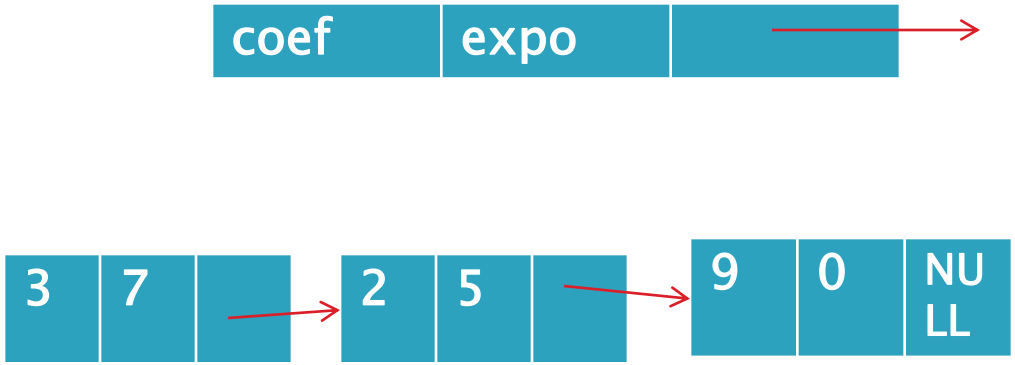
- Example Program

1. Polynomial addition, subtraction
2. RN, Name list
3. Function to check if a singly linked list is palindrome

Polynomial representation

A-> $3x^7+2x^5+9$
B-> $5x^2+ 7$

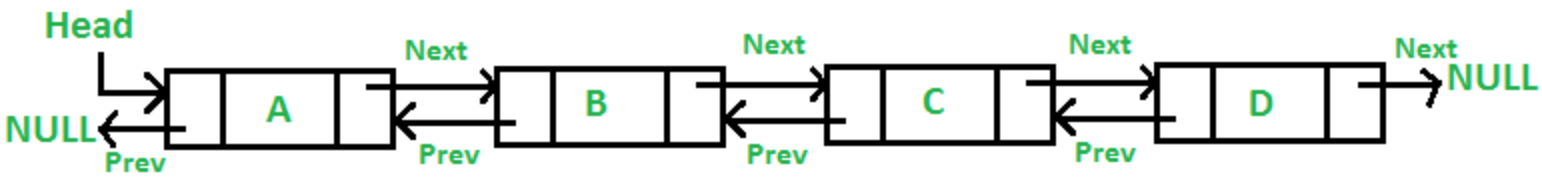
C=A+B




```
struct Node
{
  int coef;
int expo;
  struct Node *next;
};
```

Doubly Linked list

A **D**oubly **L**inked **L**ist (DLL) contains an extra pointer, typically called *previous pointer*, together with next pointer and data which are there in singly linked list.




```
/* Node of a doubly linked list */
struct Node {
    int data;
    struct Node* next; // Pointer to next node in DLL
    struct Node* prev; // Pointer to previous node in DLL
};
```

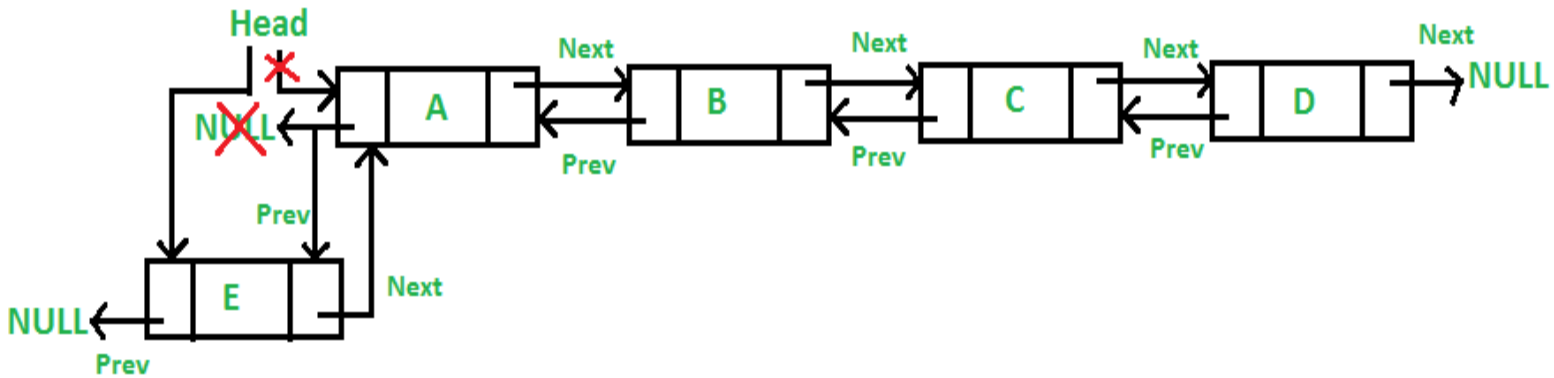
- **Advantages over singly linked list**
  1) A DLL can be traversed in both forward and backward direction.
  2) The delete operation in DLL is more efficient if pointer to the node to be deleted is given.
  3) We can quickly insert a new node before a given node. In singly linked list, to delete a node, pointer to the previous node is needed. To get this previous node, sometimes the list is traversed. In DLL, we can get the previous node using previous pointer.
- **Disadvantages over singly linked list**
  1) Every node of DLL Require extra space for an previous pointer. It is possible to implement DLL with single pointer though.
  2) All operations require an extra pointer previous to be maintained. For example, in insertion, we need to modify previous pointers together with next pointers. For example in following functions for insertions at different positions, we need 1 or 2 extra steps to set previous pointer.

```
void printList(struct Node* head)
{
    struct Node* node;
node= head;
    printf("\nTraversal in forward direction \n");
    while (node != NULL) {
        printf(" %d ", node->data);
        last = node;
        node = node->next;
    }
 node=last;
    printf("\nTraversal in reverse direction \n");
    while (node != NULL) {
        printf(" %d ", last->data);
        last = last->prev;
    }
}
```

Insertion

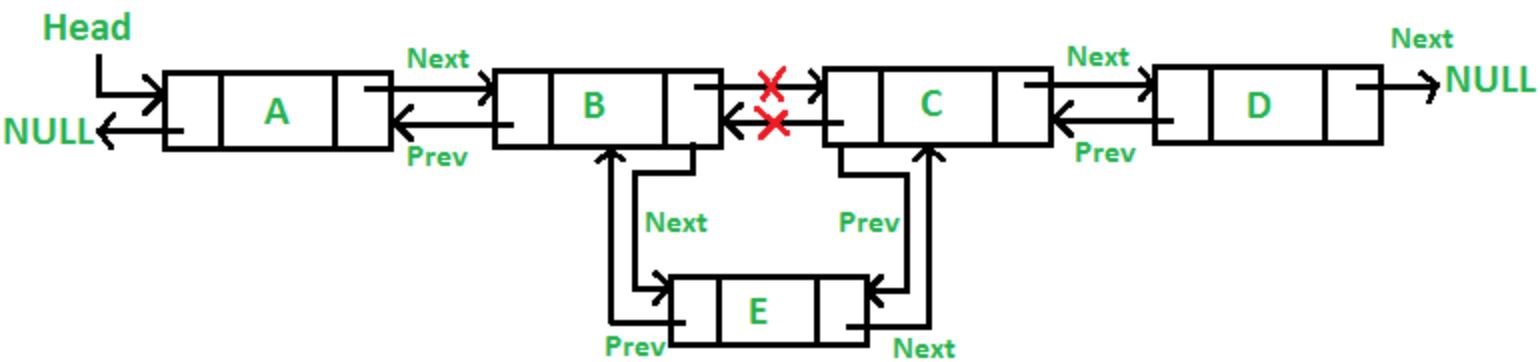
1) At the front of the DLL
2) After a given node.
3) At the end of the DLL
4) Before a given node.

- at the front


Head
NULL
A
Next
Prev
B
Next
Prev
C
Next
Prev
D
Next
NULL
Prev
E
Next
NULL
Prev

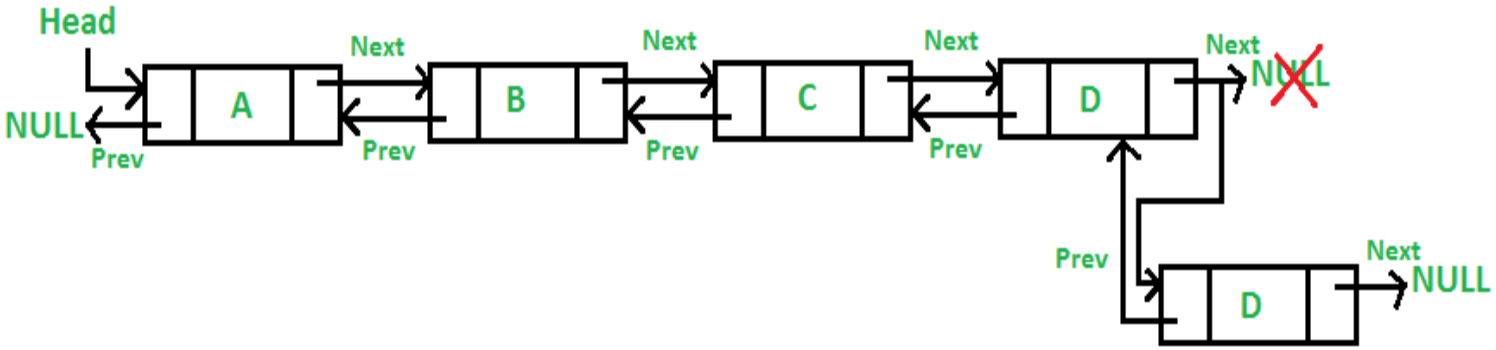
```
void push(struct Node* head, int new_data)
{
    /* 1. allocate node */
    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));

    /* 2. put in the data  */
    new_node->data = new_data;

    /* 3. Make next of new node as head and previous as NULL */
    new_node->next = (head);
    new_node->prev = NULL;

    /* 4. change prev of head node to new node */
    if ((head) != NULL)
        (head)->prev = new_node;

    /* 5. move the head to point to the new node */
    head= new_node;
}
```

- after a given node



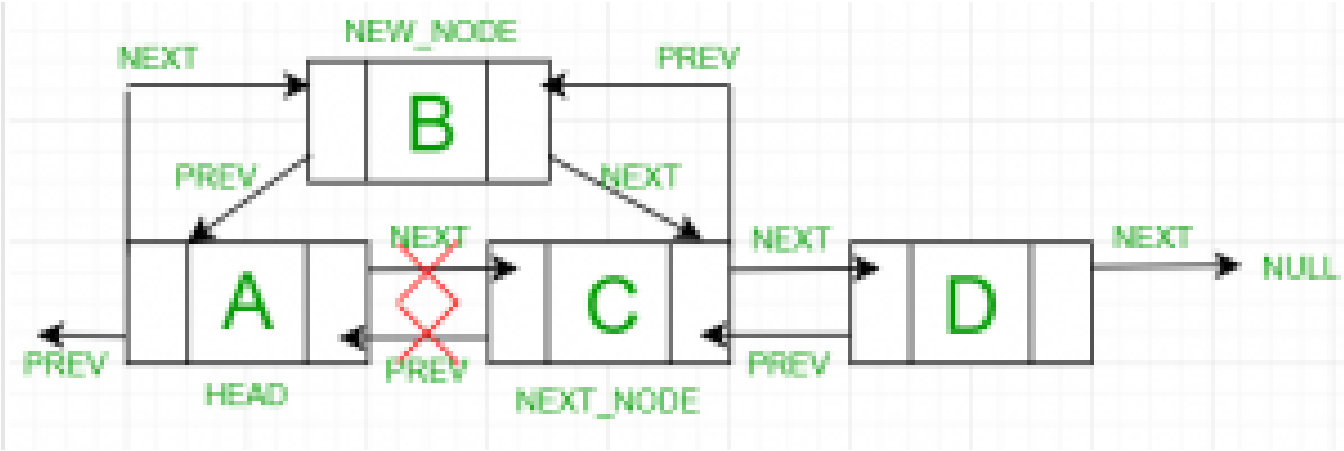
```
void insertAfter(struct Node* prev_node, int new_data)
{
    /*1. check if the given prev_node is NULL */
    if (prev_node == NULL) {
        printf("the given previous node cannot be NULL");
        return;
    }

    /* 2. allocate new node */
    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));

    /* 3. put in the data  */
    new_node->data = new_data;

    /* 4. Make next of new node as next of prev_node */
    new_node->next = prev_node->next;

    /* 5. Make the next of prev_node as new_node */
    prev_node->next = new_node;

    /* 6. Make prev_node as previous of new_node */
    new_node->prev = prev_node;

    /* 7. Change previous of new_node's next node */
    if (new_node->next != NULL)
        new_node->next->prev = new_node;
}
```

at the end


Head
NULL
Prev
A
Next
Prev
B
Next
Prev
C
Next
Prev
D
Next
NULL
Prev
D
Next
NULL

```
void append(struct Node** head_ref, int new_data)
{
    /* 1. allocate node */
    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));

    struct Node* last = *head_ref; /* used in step 5*/

    /* 2. put in the data  */
    new_node->data = new_data;

    /* 3. This new node is going to be the last node, so
          make next of it as NULL*/
    new_node->next = NULL;

    /* 4. If the Linked List is empty, then make the new
          node as head */
    if (*head_ref == NULL) {
        new_node->prev = NULL;
        *head_ref = new_node;
        return;
    }

    /* 5. Else traverse till the last node */
    while (last->next != NULL)
        last = last->next;

    /* 6. Change the next of last node */
    last->next = new_node;

    /* 7. Make last node as previous of new node */
    new_node->prev = last;

    return;
}
```

before a given node



```
void insertBefore(struct Node* next_node, int new_data)
{
    /*1. check if the given new_node is NULL */
    if (next_node == NULL) {
        printf("the given next node cannot be NULL");
        return;
    }

    /* 2. allocate new node */
    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));

    /* 3. put in the data */
    new_node->data = new_data;

    /* 4. Make prev of new node as prev of next_node */
    new_node->prev = next_node->prev;

    /* 5. Make the prev of next_node as new_node */
    next_node->prev = new_node;

    /* 6. Make next_node as next of new_node */
    new_node->next = next_node;

    /* 7. Change next of new_node's previous node */
    if (new_node->prev != NULL)
        new_node->prev->next = new_node;
}
```

```
void insertAfter(struct Node* prev_node, int new_data)
{
    /*1. check if the given prev_node is NULL */
    if (prev_node == NULL) {
        printf("the given previous node cannot be NULL");
        return;
    }

    /* 2. allocate new node */
    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));

    /* 3. put in the data  */
    new_node->data = new_data;

    /* 4. Make next of new node as next of prev_node */
    new_node->next = prev_node->next;

    /* 5. Make the next of prev_node as new_node */
    prev_node->next = new_node;

    /* 6. Make prev_node as previous of new_node */
    new_node->prev = prev_node;

    /* 7. Change previous of new_node's next node */
    if (new_node->next != NULL)
        new_node->next->prev = new_node;
}
```